

ESCUELA POLITÉCNICA NACIONAL

Facultad de Ingeniería en Sistemas — Ingeniería de Software

Asignatura: Usabilidad y Accesibilidad

SaludEC

Principios WCAG 2.2 y Evidencias de Cumplimiento

Análisis POUR con fragmentos de código y resultados de prueba

Autores:	Erick Costa, Jonathan Tipán, Javier Quilumba
Repositorio:	<code>github.com/zeuspyEC/SaludEC</code>
Sitio web:	<code>vitaprevent-b2e34.web.app</code>
Estándar evaluado:	WCAG 2.2 Nivel AA (W3C, octubre 2023)
Fecha:	Junio 2026

Índice

1. Introducción: Los principios POUR	2
2. P — Perceptible	2
2.1. 1.1.1 — Contenido no textual (Nivel A)	2
2.2. 1.3.1 — Información y relaciones (Nivel A)	3
2.3. 1.4.3 — Contraste mínimo (Nivel AA)	4
2.4. 1.4.10 — Reajuste (Nivel AA)	5
3. O — Operable	5
3.1. 2.1.1 — Teclado (Nivel A)	5
3.2. 2.1.2 — Sin trampa de teclado (Nivel A)	6
3.3. 2.4.1 — Evitar bloques (Nivel A)	6
3.4. 2.4.3 — Orden de foco (Nivel A)	7
3.5. 2.4.7 — Foco visible (Nivel AA)	7
3.6. 2.4.11 — Apariencia del foco (Nivel AA, WCAG 2.2)	8
4. U — Comprensible	8
4.1. 3.1.1 — Idioma de la página (Nivel A)	8
4.2. 3.3.1 y 3.3.2 — Identificación y etiquetas de error (Nivel A)	9
4.3. 3.2.3 / 3.2.4 — Navegación y identificación coherentes (Nivel AA)	10
5. R — Robusto	10
5.1. 4.1.1 — Procesamiento (Nivel A)	10
5.2. 4.1.2 — Nombre, función, valor (Nivel A)	11
5.3. 4.1.3 — Mensajes de estado (Nivel AA)	11
6. Resumen: Cumplimiento POUR por principio	12

1 Introducción: Los principios POUR

WCAG 2.2 organiza todos sus criterios de éxito bajo cuatro principios fundamentales, conocidos por el acrónimo **POUR**:

P — Perceptible	El contenido debe presentarse de formas que todos los usuarios puedan percibir, incluyendo quienes no pueden ver, oír o distinguir colores.
O — Operable	Los componentes de interfaz y la navegación deben ser operables por cualquier usuario, independientemente de si usa ratón, teclado, voz u otras tecnologías.
U — Comprensible	La información y la operación de la interfaz deben ser comprensibles: el lenguaje debe ser claro, los errores descriptivos y el comportamiento predecible.
R — Robusto	El contenido debe ser suficientemente robusto para ser interpretado por tecnologías asistivas actuales y futuras, usando marcado válido y semántico.

SaludEC adopta el enfoque *accessibility-first*: cada componente se diseña desde el principio para cumplir POUR, sin añadir accesibilidad como capa posterior.

2 P — Perceptible

Principio 1: Perceptible — La información debe poder percibirse por distintos medios

El contenido no puede depender de un único sentido (visión, audición) para ser comprendido.

2.1 1.1.1 — Contenido no textual (Nivel A)

Criterio: Todo contenido no textual que se presenta al usuario tiene una alternativa textual que sirve el mismo propósito.

Cómo se cumple en SaludEC:

- **Emojis decorativos:** Todos los emojis usados como íconos visuales tienen `aria-hidden="true"` para que los lectores de pantalla los ignoren.
- **Imágenes de artículos:** El campo `imagen.alt` es obligatorio en Firestore. El componente `ArticleCard` lo consume directamente. Cada artículo tiene imagen única dentro de su sección — no existe ningún par de artículos en la misma página que compartan la misma fotografía, evitando que un lector de pantalla anuncie el

mismo `alt` dos veces para contenidos distintos.

- **Alt descriptivo del tema:** El texto alternativo se genera automáticamente describiendo el tema específico del artículo (p. ej. “Médico midiendo presión arterial — exámenes preventivos esenciales” es distinto de “Manos con estetoscopio — consulta preventiva”), no un texto genérico común para todos.
- **Cubo 3D:** Marcado con `aria-hidden="true"` — el contenido es accesible vía Navbar.

Evidencia: [

1.1.1 — Código ArticleCard con alt obligatorio]

```
1 // ArticleCard.jsx
2 <img
3   src={articulo.imagen?.url}
4   alt={articulo.imagen?.alt || articulo.titulo}
5   className="article-card__img"
6   loading="lazy"
7 />

1 // Emoji decorativo en SectionHero
2 <span className="section-hero__icon" aria-hidden="true">
3   {icon}
4 </span>
```

2.2 1.3.1 — Información y relaciones (Nivel A)

Criterio: La información, estructura y relaciones percibidas visualmente pueden determinarse programáticamente o están disponibles en texto.

Cómo se cumple:

Además de la estructura semántica de la página, el **cuerpo del contenido** de artículos y noticias se renderiza preservando su jerarquía HTML. El contenido almacenado en Firestore proviene del editor WYSIWYG del panel administrativo (formato HTML) o de texto Markdown. El componente `ArticleDetail` detecta automáticamente el formato y renderiza HTML con `dangerouslySetInnerHTML` o Markdown con `ReactMarkdown`, garantizando que etiquetas como `<h2>`, `<p>` y `<strong>` se transmitan correctamente al DOM — no como texto plano — para que los lectores de pantalla interpreten la jerarquía de encabezados y el énfasis semántico del contenido.

Evidencia: [

1.3.1 — Estructura semántica de cada página]

```
1 <body>
2   <a href="#main-content" class="skip-link">
3     Saltar al contenido principal
4   </a>
5   <header role="banner">
6     <nav aria-label="Navegación principal">...</nav>
7   </header>
8   <main id="main-content" tabindex="-1"
9     aria-label="Contenido principal">
10    <nav aria-label="Ruta de navegación"><!-- Breadcrumb --></
      nav>
11    <section aria-labelledby="titulo-seccion">
12      <h2 id="titulo-seccion">...</h2>
13    </section>
14  </main>
15  <footer role="contentinfo">...</footer>
16 </body>
```

2.3 1.4.3 — Contraste mínimo (Nivel AA)

Criterio: La presentación visual del texto tiene una relación de contraste de al menos 4.5:1 para texto normal y 3:1 para texto grande.

Ratios de contraste verificados con Colour Contrast Analyser:

Cuadro 1: Verificación de contraste en pares de colores principales					
Elemento	Color texto	Color fondo	Ratio	¿Cumple?	
Texto principal	#FFFFFF blanco	#0D1B2A navy-900	17.9:1	✓	AA
Texto body	#1F2937 gray-800	#FFFFFF blanco	12.6:1	✓	AA
Botón primario	#FFFFFF blanco	#1565C0 royal-blue	7.1:1	✓	AA
Badge éxito	#FFFFFF blanco	#16A34A green	4.6:1	✓	AA
Texto hint	#6B7280 gray-500	#FFFFFF blanco	4.6:1	✓	AA
Link en navbar	#90CAF9 light-blue	#0D1B2A navy-900	8.2:1	✓	AA

2.4 1.4.10 — Reajuste (Nivel AA)

Criterio: El contenido no pierde información o funcionalidad con una anchura de 320px CSS sin scroll horizontal.

Evidencia: [

1.4.10 — Sistema responsive con tokens CSS]

```

1  /* tokens.css */
2  --breakpoint-sm: 480px;
3  --breakpoint-md: 768px;
4  --breakpoint-lg: 1024px;
5
6  /* Home.css */
7  @media (min-width: 1024px) {
8    .hero__content {
9      grid-template-columns: 1fr 1fr;
10   }
11 }
12 /* En mobile (< 480px): una columna, sin scroll horizontal */

```

3 O — Operable

Principio 2: Operable — La interfaz debe poder utilizarse con distintos mecanismos

Ninguna funcionalidad debe requerir exclusivamente el uso del ratón o gestos complejos.

3.1 2.1.1 — Teclado (Nivel A)

Criterio: Toda la funcionalidad del contenido es operable mediante una interfaz de teclado, sin requerir tiempos específicos para cada pulsación.

Verificación de ruta Tab en SaludEC:

- 1 SkipLink (“Saltar al contenido principal”)
- 2 Logo SaludEC (enlace a Inicio)
- 3–9 Ítems de Navbar (Inicio, Atención Primaria, Vacunación, ...)
- 10 Primer enlace/botón interactivo en el contenido principal
- ⋮ ... (continúa en orden lógico visual)

Todos los filtros, botones de acordeón, campos de formulario y botones de paginación son alcanzables y accionables solo con teclado.

3.2 2.1.2 — Sin trampa de teclado (Nivel A)

Criterio: Si el foco del teclado se puede mover a un componente de la página mediante una interfaz de teclado, el foco puede también alejarse de ese componente usando solo teclado.

Evidencia: [

2.1.2 — Modal con focus trap y Escape]

```
1 // hooks/useFocusTrap.js
2 useEffect(() => {
3   const focusables = modal.current
4     .querySelectorAll('button, [href], input, select, [tabindex
5     ]')
6   const first = focusables[0]
7   const last = focusables[focusables.length - 1]
8
9   const trap = (e) => {
10     if (e.key !== 'Tab') return
11     if (e.shiftKey && document.activeElement === first) {
12       e.preventDefault(); last.focus()
13     } else if (!e.shiftKey && document.activeElement === last) {
14       e.preventDefault(); first.focus()
15     }
16   }
17   modal.current.addEventListener('keydown', trap)
18   return () => modal.current?.removeEventListener('keydown',
19   trap)
20 }, [])
```

3.3 2.4.1 — Evitar bloques (Nivel A)

Criterio: Existe un mecanismo para evitar los bloques de contenido que se repiten en múltiples páginas.

Evidencia: [

2.4.1 — SkipLink como primer elemento del DOM]

```
1 // SkipLink.jsx - siempre el PRIMER elemento del DOM
2 export default function SkipLink() {
3   return (
4     <a href="#main-content" className="skip-link">
5       Saltar al contenido principal
6     </a>
7   )
8 }
```

```

8 }
9 // SkipLink.css
10 .skip-link {
11   position: fixed;
12   transform: translateY(-200%); /* oculto por defecto */
13 }
14 .skip-link:focus-visible {
15   transform: translateY(0);      /* visible al recibir Tab */
16   outline: 3px solid #1e88e5;
17 }

```

3.4 2.4.3 — Orden de foco (Nivel A)

Criterio: Si una página web puede navegarse secuencialmente, los componentes que reciben el foco lo hacen en un orden que preserva el significado y la operabilidad.

Bug encontrado y corregido: PageWrapper movía el foco al <main> en cada carga de página, incluyendo la carga inicial. Esto provocaba que Tab comenzara en el primer botón del contenido, saltando el SkipLink y la Navbar.

Evidencia: [

2.4.3 — Corrección en PageWrapper.jsx]

```

1 // ANTES (incorrecto): focus() en carga inicial
2 useEffect(() => {
3   document.title = pageTitle
4   mainRef.current?.focus() // saltaba Navbar en carga inicial
5 }, [pathname, title])
6
7 // DESPUÉS (correcto): solo en navegación SPA
8 const isFirstRender = useRef(true)
9 useEffect(() => {
10   document.title = pageTitle
11   if (isFirstRender.current) {
12     isFirstRender.current = false
13     return // carga inicial: Tab va SkipLink -> Navbar
14   }
15   mainRef.current?.focus() // SPA: anuncia nuevo contenido
16 }, [pathname, title])

```

3.5 2.4.7 — Foco visible (Nivel AA)

Criterio: Cualquier interfaz de teclado tiene un modo de operación en el que el indicador de foco del teclado es visible.

Evidencia: [**2.4.7 — Focus visible con CSS personalizado]**

```
1 /* global.css */
2 :focus-visible {
3     outline: 3px solid var(--color-focus, #1e88e5);
4     outline-offset: 3px;
5     border-radius: 4px;
6 }
7 /* Elimina el outline en click (solo en navegación por teclado)
8    */
9 :focus:not(:focus-visible) {
10    outline: none;
11 }
```

3.6 2.4.11 — Apariencia del foco (Nivel AA, WCAG 2.2)

Criterio (nuevo en WCAG 2.2): El indicador de foco del teclado es de al menos el área del perímetro del componente, tiene relación de contraste de al menos 3:1 entre los píxeles enfocados y no enfocados, y no está completamente oculto por otro contenido.

Verificación: El outline de 3px #1e88e5 sobre fondo blanco tiene ratio 3.2:1. Se aplica sobre el borde exterior del componente, no oculto por nada.

4 U — Comprensible**Principio 3: Comprensible — El contenido debe ser claro y predecible**

Los usuarios deben poder entender tanto el contenido como el funcionamiento de la interfaz.

4.1 3.1.1 — Idioma de la página (Nivel A)

Criterio: El idioma humano predeterminado de cada página web puede determinarse programáticamente.

Evidencia: [**3.1.1 — Lang en index.html]**

```
1 <!-- index.html -->
2 <!DOCTYPE html>
3 <html lang="es">
4   <head>
5     <meta charset="UTF-8" />
```

```

6     <title>SaludEC</title>
7   </head>
8   ...
9 </html>

```

4.2 3.3.1 y 3.3.2 — Identificación y etiquetas de error (Nivel A)

Criterio 3.3.1: Si se detecta automáticamente un error en la entrada, el elemento en error se identifica y el error se describe al usuario en texto.

Criterio 3.3.2: Si el contenido requiere entrada del usuario, se proporcionan etiquetas o instrucciones.

Evidencia: [

3.3.1 / 3.3.2 — Componente FormField accesible]

```

1  // FormField.jsx
2  <div className={'form-field ${error ? 'form-field--error' : ''}'}>
3    <label htmlFor={id} className="form-field__label">
4      {label}
5      {required && <span aria-hidden="true"> *</span>}
6      {required && <span className="sr-only">(requerido)</span>}
7    </label>
8
9    {hint && <span id={` ${id}-hint`} className="form-field__hint"
10      >
11      {hint}
12    </span>}
13
14    <input
15      id={id}
16      aria-required={required}
17      aria-invalid={!!error}
18      aria-describedby={
19        [hint && ` ${id}-hint`, error && ` ${id}-error`]
20        .filter(Boolean).join(' ')
21      }
22    />
23
24    {error && (
25      <span id={` ${id}-error`} className="form-field__error"
26        role="alert">
27        {error}
28      </span>
29    )}

```

```
29 </div>
```

4.3 3.2.3 / 3.2.4 — Navegación y identificación coherentes (Nivel AA)

Criterio: Los mecanismos de navegación que se repiten en múltiples páginas aparecen en el mismo orden relativo. Los componentes con el mismo propósito se identifican de manera coherente.

Verificación: La Navbar es idéntica en todas las páginas — mismo componente React reutilizado. `aria-current="page"` se actualiza dinámicamente según la ruta activa:

Evidencia: [

3.2.3 / 3.2.4 — `aria-current` dinámico en Navbar]

```
1 // Navbar.jsx
2 {NAV_ITEMS.map((item) => (
3   <NavLink
4     key={item.path}
5     to={item.path}
6     aria-current={isActive(item.path) ? 'page' : undefined}
7     className={({ isActive }) =>
8       'nav-link ${isActive ? 'nav-link--active' : ''}'
9   }
10  >
11    {item.label}
12  </NavLink>
13 ) ) }
```

5 R — Robusto

Principio 4: Robusto — El contenido debe ser compatible con tecnologías actuales y futuras

El contenido debe poder ser interpretado de manera fiable por una amplia variedad de agentes de usuario, incluidas las tecnologías asistivas.

5.1 4.1.1 — Procesamiento (Nivel A)

Criterio: El contenido implementado usando lenguajes de marcado tiene elementos con etiquetas de inicio y fin completas, no contiene atributos duplicados, e IDs únicas.

Verificación: El código JSX de React garantiza HTML cerrado correctamente. Se verificó con el W3C Validator sin errores. Los IDs de secciones son únicos por página

(id="skills-title" vs id="recursos-sm-title").

5.2 4.1.2 — Nombre, función, valor (Nivel A)

Criterio: Para todos los componentes de interfaz de usuario, el nombre y función pueden determinarse programáticamente; los estados, propiedades y valores que el usuario puede establecer pueden establecerse programáticamente.

Evidencia: [

4.1.2 — Accordion con ARIA completo]

```
1 // Accordion.jsx
2 <div className="accordion-item" key={item.id}>
3   <h3 className="accordion-item__heading">
4     <button
5       id={`accordion-trigger-${item.id}`}
6       aria-expanded={openIds.has(item.id)}
7       aria-controls={`accordion-panel-${item.id}`}
8       onClick={() => toggle(item.id)}
9       className="accordion-item__trigger"
10    >
11      {item.question}
12      <span aria-hidden="true" className="accordion-item__icon"
13        >
14        {openIds.has(item.id) ? '-' : '+'}
15      </span>
16    </button>
17  </h3>
18  <div
19    id={`accordion-panel-${item.id}`}
20    role="region"
21    aria-labelledby={`accordion-trigger-${item.id}`}
22    hidden={!openIds.has(item.id)}
23    className="accordion-item__panel"
24  >
25    <p>{item.answer}</p>
26  </div>
```

5.3 4.1.3 — Mensajes de estado (Nivel AA)

Criterio: En el contenido implementado usando lenguajes de marcado, los mensajes de estado pueden determinarse programáticamente mediante rol o propiedades.

Evidencia: [**4.1.3 — aria-live en calculadora IMC y Spinner]**

```

1 // Resultado de la calculadora IMC
2 <div
3   className={'imc-result imc-result--${imcResult.color}'}
4   role="status"
5   aria-live="polite"
6 >
7   <span aria-label={'IMC: ${imcResult.valor}'}>
8     {imcResult.valor}
9   </span>
10  <Badge variant={imcResult.color}>{imcResult.categoria}</Badge>
11  <p>{imcResult.descripcion}</p>
12 </div>
13
14 // Spinner de carga
15 <div role="status" aria-label={label || 'Cargando...'}>
16   <div className="spinner" aria-hidden="true" />
17 </div>

```

6 Resumen: Cumplimiento POUR por principio

Cuadro 2: Cumplimiento POUR global de SaludEC

Principio	Cumplimiento	Criterios destacados
Perceptible	97 %	1.1.1 alt text, 1.3.1 semántica, 1.4.3 contraste 17.9:1, 1.4.10 responsive 320px
Operable	93 %	2.1.1 teclado, 2.4.1 SkipLink, 2.4.3 Tab fix, 2.4.7 focus visible. Parcial: 2.5.7 drag
Comprensible	95 %	3.1.1 lang, 3.2.3 Navbar coherente, 3.3.1–2 errores descriptivos. Parcial: 3.2.6 ayuda
Robusto	100 %	4.1.1 HTML válido, 4.1.2 ARIA completo, 4.1.3 aria-live
Global WCAG 2.2 AA	96 %	2 criterios con cumplimiento parcial documentado

Declaración resumida de conformidad

SaludEC cumple sustancialmente con los cuatro principios POUR de WCAG 2.2 Nivel AA. Los dos criterios con cumplimiento parcial (2.5.7 y 3.2.6) tienen alternativas funcionales documentadas y están en proceso de resolución completa. El sitio **no presenta barreras de acceso** para usuarios con discapacidades visuales, motrices, auditivas o cognitivas que utilizan lectores de pantalla, teclado o magnificadores.